

Engenharia de Plataforma Multilocatária para Distribuição de Gás e Água: Adaptação de Ecossistema de Pagamentos com Cashback de Alta Performance

Análise e Adaptação do Modelo Xulis para Gás e Água

A transposição de um ecossistema de pagamentos e fidelização originalmente concebido para postos de abastecimento de combustível, como o modelo da plataforma Xulis, para o setor de distribuição retalhista de gás de botija (gás liquefeito de petróleo - GLP) e garrações de água mineral exige uma reengenharia completa dos fluxos de trabalho operacionais e do modelo de negócio¹. A plataforma Xulis opera como uma solução integrada que une a pista de abastecimento, o terminal de ponto de venda (POS), a conciliação financeira automatizada e uma carteira digital com cashback instantâneo para frotas e motoristas particulares¹.

Ao nível do posto de combustível tradicional, a interação ocorre num espaço físico fixo onde o fluxo de vendas é monitorizado por frentistas através de maquininhas de pagamento inteligentes (XPay POS), as quais registam as formas de pagamento, gerem a abertura e o encerramento de caixas por turnos e reduzem divergências operacionais sem a necessidade de computadores na pista de abastecimento². O sistema de retaguarda (Xulis ERP) centraliza as vendas e o inventário em tempo real, validando os recebimentos através de uma conciliadora inteligente². Toda esta complexidade operacional é ocultada do cliente final, que usufrui de uma aplicação de pagamento de clique único, recebendo notificações instantâneas para autorizar transações com segurança, sem cartões físicos ou leitura manual de códigos QR, acumulando créditos que podem ser resgatados imediatamente em parceiros da rede³. Para reconfigurar este ecossistema para uma aplicação multilocatária (*SaaS multi-tenant*) de distribuição de gás e água⁹, onde franquias (como uma unidade da Ultragaz) operam como inquilinos (*tenants*) do sistema, a arquitetura deve abandonar o modelo de ponto físico centralizado e adotar uma logística de distribuição descentralizada e móvel¹⁰. A venda direta de gás e água requer processos de agendamento, roteirização geográfica de frotas e validações de entrega ao domicílio que não existem na pista de um posto de combustível¹¹. O quadro abaixo estabelece a equivalência técnica e operacional entre os recursos originais do ecossistema Xulis e a sua adaptação para a plataforma de distribuição de gás e água:

Funcionalidade Original	Característica	Adaptação para Gás e
-------------------------	----------------	----------------------

(Xulis)	Operacional (Postos)	Água (Multilocatário)
Abastecimento e Venda de Pista [cite: 2, 6]	Integração com bombas de combustível físicas e automação de pista.	Compra Direta na Aplicação: Carrinho de compras com seleção de botijas (P13, P45) e garrações de água (20L, 10L) com entrega imediata ou agendada.
Maquininha de Pagamento XPay [cite: 2, 5]	Terminal POS portátil de pista integrado ao ERP para frentistas.	Driver App / Mobile POS: Aplicação do entregador com suporte para transações de cartão e PIX no local de entrega, sincronizando o inventário do veículo em tempo real.
Cashback Instantâneo [cite: 3, 7, 8]	Crédito instantâneo na carteira digital utilizável em postos parceiros.	Cashback por Franquia: Saldo acumulado específico para cada franquia inquilina, incentivando a fidelização e a recorrência cíclica de compra na mesma unidade.
Conciliação Automatizada 4.0 [cite: 2, 6]	Cruzamento de taxas de operadoras e conciliação de vendas na pista.	Conciliação de Cilindros e Caixa: Monitorização e correspondência automática de cilindros cheios entregues vs. botijas vazias recolhidas e conciliação de caixa.
BPO Financeiro via WhatsApp ("Xuxu") [cite: 2, 4]	Monitorização de contas e pagamentos automáticos coordenados por Inteligência Artificial via chat.	Predição de Consumo e Reencomenda Automatizada: Análise preditiva baseada no tempo médio de esgotamento de gás/água, disparando avisos e links de compra no WhatsApp.

Controlo de Turnos e Operadores [cite: 2, 5]	Auditoria detalhada de vendas por frentista, turno e terminal.	Rastreamento e Despacho de Frotas: Roteirização inteligente baseada na carga disponível nos veículos em trânsito e atribuição automática ao entregador mais próximo.
--	--	--

A grande inovação desta conversão reside no facto de a plataforma não pertencer exclusivamente a uma revendedora, mas funcionar como um modelo de *Software as a Service* (SaaS) multilocatário⁹. A franquia do utilizador é o primeiro cliente de grande relevância, mas o sistema está arquitetado para que qualquer outro distribuidor de gás ou água possa registar-se como inquilino, personalizar as suas taxas de cashback, gerir a sua própria frota de entregadores e manter o isolamento absoluto dos seus dados financeiros e de clientes⁹. Ao converter o ecossistema de inteligência e o módulo financeiro baseado em conversas de WhatsApp, a IA "Xuxu" assume um papel preditivo². Para o mercado de gás de botija e garrações de água, a compra é altamente cíclica e previsível¹¹. Em vez de o utilizador gerir apenas as contas a pagar pelo WhatsApp², o agente inteligente analisa a frequência histórica de consumo do cliente¹¹. Por exemplo, se uma família consome um cilindro P13 a cada quarenta dias, a IA envia uma notificação interativa pelo WhatsApp ao trigésimo oitavo dia¹⁰: "Olá! O seu gás deve estar quase no fim. Deseja agendar a entrega para amanhã às 14h com aplicação automática de 15€ do seu saldo de cashback acumulado?"². Este nível de serviço eleva o relacionamento com o cliente e reduz a taxa de evasão de clientes (*churn*) para as franquias parceiras⁸.

Arquitetura do Sistema e Estrutura Multilocatária

A arquitetura do sistema foi estruturada sob o padrão de monorepo utilizando o Turborepo¹⁷. O monorepo resolve os problemas clássicos de sincronização de contratos de dados em equipas ágeis, permitindo que a camada de base de dados e de tipos TypeScript seja inteiramente partilhada entre os portais Web e as aplicações móveis¹⁷.

O ecossistema é composto por quatro aplicações de ponta e pacotes de suporte partilhados:

- **Aplicação Móvel do Cliente (apps/mobile-customer):** Desenvolvida em Expo com a Nova Arquitetura (*Fabric*). Permite a compra de produtos, seleção de entrega imediata ou agendamento, gestão de moradas e consulta do saldo de cashback acumulado específico do distribuidor em uso.
- **Aplicação Móvel do Entregador (apps/mobile-driver):** Aplicação focada na operação de logística do motorista, responsável pela navegação em tempo real com mapas, atualização do estado do pedido e confirmação física de entrega através de prova fotográfica e geofencing.
- **Painel Administrativo do Franqueado (apps/web-tenant-admin):** Dashboard desenvolvido em Next.js para os gestores da franquia (como o pai do utilizador)

acompanharem as encomendas em tempo real, gerirem o inventário físico de garrações e cilindros, definirem as percentagens de cashback e monitorizarem a frota de entregadores.

- **Painel de Gestão da Plataforma (apps/web-platform-super):** Painel de controlo centralizado do proprietário do software (o utilizador) para faturamento das subscrições das franquias inquilinas, monitorização de taxas de transações globais e gestão de inquilinos (*tenants*).

```
|
├── apps
│   ├── web-tenant-admin/      # Painel Next.js 15 do Franqueado (Inquilino)
│   ├── web-platform-super/   # Painel Next.js 15 do Gestor do SaaS (Super Admin)
│   ├── mobile-customer/      # Aplicação Move! Expo para Clientes [cite: 19, 20]
│   ├── mobile-driver/        # Aplicação Move! Expo para Motoristas/Entregadores
│   └── packages
│       ├── db/               # Esquema Drizzle ORM, migrações e triggers SQL [cite: 17, 19]
│       ├── ui-web/           # Componentes Web reutilizáveis (shadcn/ui + Tailwind v4)
│       └── ui-native/        # Componentes Móveis reutilizáveis (NativeWind + Reanimated
v4
│   ├── api-client/           # Cliente Supabase tipado com funções utilitárias
│   └── package.json
└── turbo.json                # Configuração do pipeline do Turborepo [cite: 17, 18]
```

O isolamento lógico de cada revendedora é garantido diretamente na camada de persistência através de Row-Level Security (RLS) no PostgreSQL do Supabase⁹. Cada inquilino possui um identificador único universal (*tenant_id*)¹⁵. Quando um utilizador administrativo autentica-se, o seu token JWT é decorado com o seu respetivo *tenant_id* nos metadados da aplicação (*app_metadata*) através de um gatilho de autenticação (*Auth Hook*)¹⁵. Nenhuma query enviada ao Supabase necessita de cláusulas manuais de filtragem de inquilino na aplicação, eliminando o risco de vazamento de dados devido a falhas de desenvolvimento na camada de visualização⁹.

O quadro seguinte descreve a matriz de acessibilidade e escopo de cada aplicação em execução no monorepo:

Aplicação	Público-Alvo	Stack Principal	Mecanismo de Isolamento de
-----------	--------------	-----------------	----------------------------

			Dados
web-platform-super	Dono do Software	Next.js 15 (SSR)	Chave de serviço seguro (<i>service_role</i>) com bypass de RLS para gestão global.
web-tenant-admin	Franqueado / Gestor	Next.js 15 (Hybrid)	RLS rigoroso com base no <i>tenant_id</i> injetado no JWT de autenticação.
mobile-customer	Consumidor Final	Expo Router (Fabric)	RLS limitado ao ID do próprio utilizador (<i>auth.uid()</i>) e visualização livre de produtos.
mobile-driver	Motorista	Expo Router (Fabric)	RLS que expõe apenas pedidos atribuídos ao ID do motorista ou pendentes no seu inquilino.

Stack Tecnológica e Engenharia de Interface Fluida

Para entregar uma experiência de alto nível (*premium*), leve e de resposta instantânea para o utilizador, a stack tecnológica adota o estado da arte do desenvolvimento web e móvel. A escolha das tecnologias foca-se na eliminação de gargalos que historicamente degradam a experiência de aplicações multiplataforma, como a latência de renderização da interface e as barreiras assíncronas de comunicação entre o JavaScript e as linguagens nativas dos dispositivos móveis.

A interface móvel baseia-se no Expo utilizando o React Native na sua Nova Arquitetura (*Fabric*) com o modo *Bridgeless* ativo²⁰. Diferente do modelo legado que dependia do envio de mensagens JSON serializadas através de uma ponte assíncrona — o que gerava quedas de frames e bloqueios em animações complexas —, a arquitetura *Fabric* utiliza a JavaScript Interface (JSI)²¹. A JSI permite a exposição direta de APIs escritas em C++ (nativas do iOS e Android) para o motor JavaScript do dispositivo móvel²¹. Desta forma, a manipulação de vistas nativas e estados de interface gráfica ocorre de forma síncrona, eliminando atrasos perceptíveis nas interações de toque mais exigentes, como a navegação fluida em mapas¹⁰.

As animações são governadas pelo **React Native Reanimated v4**. Esta biblioteca executa transições e cálculos de movimento em tempo real diretamente na thread nativa de UI (User Interface thread), isolando o comportamento visual do estado de processamento da thread principal do JavaScript. Se o JavaScript estiver ocupado a desserializar dados volumosos de uma consulta ao Supabase, a animação do mapa de rastreamento ou a transição dos ecrãs de checkout mantém-se a 60 ou 120 FPS estáveis, prevenindo engasgos visuais.

O Reanimated v4 introduz duas frentes distintas de manipulação de estados de interface para o programador:

- **Transições Declarativas CSS:** Ideais para animações de estado simples, como expansão de botões de compra, surgimento de balões de cashback e estados de carregamento²¹. O programador define as propriedades a animar num vetor e as mudanças de estado do React acionam transições fluidas automáticas geridas pelo motor de renderização nativo²¹.
- **Worklets de Baixo Nível para Gestos Complexos:** Utilizados em interações orientadas por toque contínuo, como o arraste da gaveta inferior (*bottom sheet*) de rastreamento de entregas ou gestos de deslize para confirmar a compra (*slide to action*)²¹. Os worklets são funções JavaScript compiladas para correr diretamente na thread de UI, respondendo instantaneamente a eventos capturados pela biblioteca `react-native-gesture-handler`²¹.

A renderização estilizada é padronizada com **Tailwind CSS v4** (via NativeWind v4 no ambiente móvel)¹⁹. O novo motor do Tailwind CSS v4 foi inteiramente reescrito em Rust, otimizando o processo de compilação estática e gerando folhas de estilo ultraleves¹⁹. No React Native, o NativeWind v4 compila as classes utilitárias para estilos nativos otimizados durante o build da aplicação, reduzindo o custo computacional de interpretação dinâmica de estilos em tempo de execução no dispositivo do utilizador.

Engenharia de Dados com Supabase: Modelagem e Segurança Baseada em RLS

O backend do sistema apoia-se inteiramente no Supabase (PostgreSQL), tirando partido da sua infraestrutura de tempo real e segurança ao nível da linha⁹. O Supabase automatiza a criação de APIs REST e conexões WebSockets para atualizações instantâneas baseadas nas tabelas de dados, permitindo manter os ecrãs de localização e estado da encomenda constantemente atualizados sem necessidade de pooling HTTP manual⁹.

Estratégia de Isolamento de Dados Multilocatários (RLS)

Para assegurar que o isolamento entre franqueados concorrentes seja inviolável, implementa-se uma arquitetura baseada no princípio do menor privilégio utilizando políticas RLS do Postgres combinadas com custom claims no JWT de autenticação⁹. O uso de subqueries em políticas RLS para verificar se o utilizador pertence a um inquilino gera graves problemas de desempenho à medida que o volume de transações cresce, dado que a subquery é executada para cada linha analisada pela consulta principal¹⁵.

Para resolver este gargalo com complexidade temporal constante $O(1)$ ³³:

1. Utiliza-se um gatilho de autenticação do Supabase (*Custom Access Token Hook*) que intercede imediatamente antes da emissão de cada token JWT pelo GoTrue²⁵.
2. O hook consulta a tabela *profiles* uma única vez no momento do login e injeta as variáveis *tenant_id* e *user_role* diretamente nos metadados da aplicação do JWT (*app_metadata*)²⁴.
3. As políticas RLS de todas as tabelas transacionais e operacionais passam a consumir estes dados diretamente da memória utilizando a função *auth.jwt()*²⁵.
4. Adicionalmente, otimiza-se o planeador de consultas do Postgres encapsulando a chamada *auth.jwt()* numa instrução *SELECT* direta²³. O padrão (*SELECT (auth.jwt() -> 'app_metadata' -> 'tenant_id')::uuid*) instrui o motor a avaliar e guardar o resultado em cache uma única vez para toda a transação, em vez de avaliar a função a cada linha da tabela²³.
5. Todas as colunas utilizadas para filtragem de políticas, especialmente *tenant_id* e *customer_id*, são indexadas utilizando árvores estruturadas do tipo B-Tree para máxima performance em volumes massivos de dados¹⁵.
6. Para garantir a imutabilidade e a conformidade do RLS em ambientes de produção, instala-se um gatilho de eventos ao nível do sistema que força a ativação automática do Row-Level Security em qualquer nova tabela criada no esquema público, mitigando falhas humanas de implantação²³.

Gestão Matemática de Cashback e Transações

O controlo do passivo financeiro de cashback emitido pelas franquias exige precisão absoluta nas transações³⁵. Para mitigar imprecisões e erros de arredondamento decorrentes de operações com pontos flutuantes em computação monetária, todos os valores financeiros da plataforma são estritamente registados em cêntimos (números inteiros)¹³.

A integridade do saldo atual de cashback de um cliente num determinado inquilino é dada pela soma do histórico de movimentações guardadas na tabela *cashback_ledger*. O saldo utilizável

consolidado $S_{utilizador}$ de um cliente num inquilino específico é computado matematicamente pela soma algébrica de todas as suas movimentações de acumulação e resgate³⁵:

$$S_{utilizador} = \sum_{i=1}^N M_i$$

Onde $M_i \in \mathbb{Z}$ representa o valor em cêntimos da movimentação transacional i para o cliente no respetivo inquilino³⁵. O saldo total nunca pode assumir um valor negativo, sendo esta barreira imposta diretamente por uma restrição de verificação (*CHECK constraint*) na base de dados:

$$S_{utilizador} \geq 0$$

Para um pedido de valor bruto de compra V_{bruto} (em cêntimos), o cliente pode optar por resgatar um montante de cashback acumulado $C_{resgate}$ de forma a amortizar a fatura³⁵. O valor líquido a ser pago pelo cliente ($V_{liquido}$) é determinado pela seguinte equação³⁵:

$$V_{liquido} = V_{bruto} - C_{resgate}$$

Onde $C_{resgate}$ está limitado pela regra de negócio do inquilino e pelo saldo real do cliente no momento da transação:

$$C_{resgate} \leq S_{utilizador}$$

O novo cashback gerado pela compra atual (C_{gerado}) que entrará na conta do cliente em estado pendente até a confirmação de entrega física é calculado individualmente por item de pedido de forma a suportar campanhas promocionais distintas para gás e água³⁵:

$$C_{gerado} = \sum_{k=1}^P \left[V_{item,k} \times \frac{\gamma_{product,k}}{100} \right]$$

Onde $V_{item,k}$ é o valor líquido do item k faturado no pedido, $\gamma_{product,k}$ é o percentual de cashback definido para aquele produto específico pelo franqueado e $\lfloor \dots \rfloor$ representa a função piso para garantir o arredondamento para o cêntimo inteiro inferior mais próximo, evitando a geração fracionária de crédito financeiro³⁵.

Especificação Técnica em Lógica e Código

Esta secção expõe os esqueletos de código limpos e funcionais, sem supressões ou atalhos de desenvolvimento, necessários para erguer a fundação da plataforma de distribuição¹⁷.

1. Migração Inicial da Base de Dados (SQL DDL com Índices, Triggers e RLS)

Este script SQL configura as tabelas de dados, os índices espaciais GiST e de busca B-Tree, habilita o Row-Level Security e cria um gatilho para garantir que novas tabelas não escapem sem políticas de isolamento ativas¹⁰.

SQL

-- Habilitar extensões geoespaciais e de criptografia

```
CREATE EXTENSION IF NOT EXISTS postgis SCHEMA public;
```

```
CREATE EXTENSION IF NOT EXISTS uuid-ossp SCHEMA public;
```

-- Definição dos Tipos Enums

```
CREATE TYPE public.user_role AS ENUM ('super_admin' 'tenant_admin' 'driver' 'customer');
```

```
CREATE TYPE public.order_status AS ENUM ('pending' 'accepted' 'in_transit' 'delivered' 'cancelled');
```

```
CREATE TYPE public.payment_method AS ENUM ('pix' 'credit_card' 'cash_on_delivery');
```

-- 1. TABELA DE INQUILINOS (TENANTS/FRANQUIAS)

```
CREATE TABLE public.tenants
```

```
(id UUID PRIMARY KEY DEFAULT uuid_generate_v4());
```

```
name VARCHAR(255) NOT NULL;
```

```
email VARCHAR(14) UNIQUE NOT NULL;
```

```
active BOOLEAN DEFAULT TRUE NOT NULL;
```

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', text, now()) NOT NULL;
```

```
);
```

```
ALTER TABLE public.tenants ENABLE ROW LEVEL SECURITY;
```

-- 2. TABELA DE PERFIS DE UTILIZADORES

```
CREATE TABLE public.profiles
```

```
(id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE);
```

```
tenant_id UUID REFERENCES public.tenants(id) ON DELETE SET NULL;
```

```
role public.user_role DEFAULT 'customer' public.user_role NOT NULL;
```

```
full_name VARCHAR(255) NOT NULL;
```

```
phone VARCHAR(20) NOT NULL;
```

```
created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', text, now()) NOT NULL;
```

```
);
```

```
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
```

-- 3. TABELA DE PRODUTOS (Botijas de Gás P13/P45, Garrações de Água 20L)

```
CREATE TABLE public.products
```

```
(id UUID PRIMARY KEY DEFAULT uuid_generate_v4());
```

```
tenant_id UUID REFERENCES public.tenants(id) ON DELETE CASCADE NOT NULL;
```

```

name VARCHAR 255 NOT NULL
description TEXT
price_cents INTEGER NOT NULL -- Armazenado em cêntimos para precisão matemática
cashback_percent NUMERIC 5 2 DEFAULT 0.00 NOT NULL
stock_quantity INTEGER DEFAULT 0 NOT NULL
active BOOLEAN DEFAULT TRUE NOT NULL
created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', text, now()) NOT NULL

```

```

ALTER TABLE public.products ENABLE ROW LEVEL SECURITY

```

```

-- 4. TABELA DE ENCOMENDAS (ORDERS)

```

```

CREATE TABLE public.orders
(id UUID PRIMARY KEY DEFAULT uuid_generate_v4())
tenant_id UUID REFERENCES public.tenants(id) ON DELETE CASCADE NOT NULL
customer_id UUID REFERENCES public.profiles(id) NOT NULL
driver_id UUID REFERENCES public.profiles(id)
status public.order_status DEFAULT 'pending' public.order_status NOT NULL
payment_method public.payment_method NOT NULL
delivery_address TEXT NOT NULL
delivery_location public.geometry(Point, 4326) -- PostGIS Geo-coordenadas
total_cents INTEGER NOT NULL
cashback_applied_cents INTEGER DEFAULT 0 NOT NULL
cashback_earned_cents INTEGER DEFAULT 0 NOT NULL
scheduled_for TIMESTAMP WITH TIME ZONE
otp_code VARCHAR 6 NOT NULL -- Chave OTP gerada para validação física da entrega
created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', text, now()) NOT NULL

```

```

ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY

```

```

-- 5. TABELA DE ITENS DE ENCOMENDA

```

```

CREATE TABLE public.order_items
(id UUID PRIMARY KEY DEFAULT uuid_generate_v4())
order_id UUID REFERENCES public.orders(id) ON DELETE CASCADE NOT NULL
product_id UUID REFERENCES public.products(id) NOT NULL
quantity INTEGER NOT NULL
price_cents INTEGER NOT NULL

```

```

ALTER TABLE public.order_items ENABLE ROW LEVEL SECURITY

```

```

-- 6. TABELA DE HISTÓRICO DE TRANSAÇÕES DE CASHBACK (LEDGER)

```

```
CREATE TABLE public.cashback_ledger
(
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID REFERENCES public.tenants(id) ON DELETE CASCADE NOT NULL,
    customer_id UUID REFERENCES public.profiles(id) NOT NULL,
    order_id UUID REFERENCES public.orders(id) ON DELETE SET NULL,
    amount_cents INTEGER NOT NULL -- Positivo para ganho, negativo para consumo [cite: 8, 35]
    description TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', text_now()) NOT NULL
);
```

```
ALTER TABLE public.cashback_ledger ENABLE ROW LEVEL SECURITY;
```

```
-- 7. TABELA DE LOCALIZAÇÃO DO ENTREGADOR EM TEMPO REAL
```

```
CREATE TABLE public.driver_locations
(
    driver_id UUID PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
    tenant_id UUID REFERENCES public.tenants(id) ON DELETE CASCADE NOT NULL,
    location public.geometry(Point, 4326) NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc', text_now()) NOT NULL
);
```

```
ALTER TABLE public.driver_locations ENABLE ROW LEVEL SECURITY;
```

```
-- =====
-- CONSTRUÇÃO DE ÍNDICES DE ALTA PERFORMANCE
-- =====
```

```
CREATE INDEX ix_profiles_tenant ON public.profiles USING btree (tenant_id) WHERE
tenant_id IS NOT NULL;
CREATE INDEX ix_products_tenant_active ON public.products USING btree (tenant_id,
active);
CREATE INDEX ix_orders_tenant_status ON public.orders USING btree (tenant_id, status);
CREATE INDEX ix_orders_customer ON public.orders USING btree (customer_id);
CREATE INDEX ix_orders_driver ON public.orders USING btree (driver_id) WHERE driver_id IS
NOT NULL;
CREATE INDEX ix_cashback_ledger_lookup ON public.cashback_ledger USING btree
(customer_id, tenant_id);
CREATE INDEX ix_driver_locations_spatial ON public.driver_locations USING gist (location);
-- Índice espacial GiST para PostGIS
```

```
-- =====
-- FUNÇÕES DE SUPORTE À SEGURANÇA (RLS CONTEXT) [cite: 23, 24]
-- =====
```

```
CREATE OR REPLACE FUNCTION public.get_auth_tenant_id()
```

```

RETURNS UUID AS $$
-- Execução otimizada em cache via SELECT de uma instrução avaliada uma única vez
SELECT NULLIF(current_setting('request.jwt.claims' true) jsonb -> 'app_metadata' ->>
'tenant_id' " )::uuid;
$$ LANGUAGE sql STABLE SECURITY DEFINER SET search_path = "

```

```

CREATE OR REPLACE FUNCTION public.get_auth_role()
RETURNS public.user_role AS $$
SELECT NULLIF(current_setting('request.jwt.claims' true) jsonb -> 'app_metadata' ->>
'user_role' " )::public.user_role;
$$ LANGUAGE sql STABLE SECURITY DEFINER SET search_path = "

```

```

-- =====
-- POLÍTICAS DE ROW-LEVEL SECURITY (RLS)
-- =====

```

```

-- POLÍTICAS: PRODUTOS
CREATE POLICY 'Clientes e utilizadores leem produtos atóis de qualquer loja' ON
public.products
FOR SELECT TO authenticated
USING (active = true)

```

```

CREATE POLICY 'Administradores de inquilino gerem produtos da sua loja' ON
public.products
FOR ALL TO authenticated
USING (tenant_id = SELECT public.get_auth_tenant_id() ) AND SELECT
public.get_auth_role() = 'tenant_admin'::public.user_role;
WITH CHECK (tenant_id = SELECT public.get_auth_tenant_id() ) AND SELECT
public.get_auth_role() = 'tenant_admin'::public.user_role;

```

```

-- POLÍTICAS: ENCOMENDAS
CREATE POLICY 'Clientes visualizam as suas próprias encomendas' ON public.orders
FOR SELECT TO authenticated
USING (customer_id = SELECT auth.uid());

```

```

CREATE POLICY 'Clientes inserem as suas próprias encomendas' ON public.orders
FOR INSERT TO authenticated
WITH CHECK (customer_id = SELECT auth.uid() ) AND status =
'pending'::public.order_status);

```

```

CREATE POLICY 'Administradores gerem todas as encomendas da sua filial' ON
public.orders
FOR ALL TO authenticated

```

```

    USING (tenant_id = SELECT public.get_auth_tenant_id()) AND SELECT
public.get_auth_role() = 'tenant_admin' public.user_role;

    WITH CHECK (tenant_id = SELECT public.get_auth_tenant_id()) AND SELECT
public.get_auth_role() = 'tenant_admin' public.user_role;

```

```

CREATE POLICY 'Entregadores visualizam e operam as suas entregas' ON public.orders
    FOR ALL TO authenticated
    USING
        tenant_id = SELECT public.get_auth_tenant_id() AND
        SELECT public.get_auth_role() = 'driver' public.user_role AND (driver_id = SELECT
auth.uid() OR driver_id IS NULL
    );

    WITH CHECK
        tenant_id = SELECT public.get_auth_tenant_id() AND
        SELECT public.get_auth_role() = 'driver' public.user_role AND driver_id = SELECT
auth.uid()
    );

```

```

-- POLÍTICAS: CASHBACK LEDGER
CREATE POLICY 'Clientes veem o seu próprio extrato de cashback' ON
public.cashback_ledger
    FOR SELECT TO authenticated
    USING (customer_id = SELECT auth.uid());

```

```

CREATE POLICY 'Administradores veem transações de cashback do seu inquilino' ON
public.cashback_ledger
    FOR SELECT TO authenticated
    USING (tenant_id = SELECT public.get_auth_tenant_id() AND SELECT
public.get_auth_role() = 'tenant_admin' public.user_role);

```

```

-- TRIGGER GLOBAL: FORÇAR ATIVAÇÃO AUTOMÁTICA DE RLS PARA NOVAS TABELAS [cite: 24]
CREATE OR REPLACE FUNCTION public.force_ri_auto_enable()
    RETURNS EVENT_TRIGGER
    LANGUAGE plpgsql
    SECURITY DEFINER
    SET search_path = pg_catalog
    AS
    DECLARE
        cmd record;
    BEGIN
        FOR cmd IN SELECT * FROM pg_event_trigger_ddl_commands() WHERE command_tag IN

```

```

'CREATE TABLE' 'CREATE TABLE AS'
LOOP
    IF cmd.schema_name = 'public' THEN
        EXECUTE format('ALTER TABLE if exists %s ENABLE ROW LEVEL SECURITY'
cmd.object_identity)
        RAISE LOG 'RLS forçado com sucesso na tabela %' cmd.object_identity
    END IF
END LOOP
END
END
END

CREATE EVENT TRIGGER ensure_rls_on_all_tables
ON ddl_command_end
WHEN TAG IN 'CREATE TABLE' 'CREATE TABLE AS'
EXECUTE FUNCTION public.force_rls_auto_enabled

```

2. Supabase Custom Access Token Hook

Esta função PostgreSQL é acionada antes de o Supabase GoTrue emitir um token JWT para o cliente²⁵. Ela pesquisa as informações do utilizador na tabela profiles e injeta-as de forma segura no payload de autenticação²⁵.

```

SQL
-- Criar a função do hook de autenticação do Supabase
CREATE OR REPLACE FUNCTION public.custom_access_token_hook(event jsonb)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY DEFINER
SET search_path = "
AS
DECLARE
    current_user_id uuid;
    found_tenant_id uuid;
    found_role public.user_role;
    claims jsonb;
BEGIN
    -- Obter o ID do utilizador autenticado

```

```

current_user_id = (event ->> 'user_id').uuid

-- Obter os dados de perfil
SELECT tenant_id, role
INTO found_tenant_id, found_role
FROM public.profiles
WHERE id = current_user_id

-- Atribuição padrão se não houver perfil gerado
IF found_role IS NULL THEN
    found_role = 'customer'::public.user_role
END IF

-- Extrair nó de claims do payload [cite: 26]
claims = event -> 'claims'

-- Injetar variáveis em app_metadata (bloqueado para edição pelo utilizador direto)
claims = jsonb_set(claims, '{app_metadata, tenant_id}' to jsonb(found_tenant_id))
claims = jsonb_set(claims, '{app_metadata, user_role}' to jsonb(found_role))

-- Retornar o evento processado de volta para o gateway [cite: 26]
RETURN jsonb_set(event, '{claims}', claims)
EXCEPTION
WHEN OTHERS THEN
    RETURN event; -- Retorna evento sem modificações em caso de falha de ligação
END
$$

-- Atribuir a permissão ao utilizador administrador do Auth
GRANT EXECUTE ON FUNCTION public.custom_access_token_hook(jsonb) TO
supabase_auth_admin;

```

3. Componente de Rastreamento Nativo Móvel (Expo + Reanimated v4 + Gestures)

Este ecrã, escrito para o Expo Router com suporte à Nova Arquitetura Fabric, implementa uma gaveta inferior flutuante para rastreamento que desliza de forma reativa a comandos táteis e corre inteiramente no thread de UI nativa²¹.

TypeScript

```
// apps/mobile-customer/src/components/TrackingDrawer.tsx
import React from 'react'
import { StyleSheet, View, Text, Dimensions } from 'react-native'
import { GestureDetector, Gesture } from 'react-native-gesture-handler'
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
  interpolate,
  Extrapolation,
} from 'react-native-reanimated'

const { height: SCREEN_HEIGHT } = Dimensions.get('window')
const COLLAPSED_HEIGHT = SCREEN_HEIGHT * 0.25
const EXPANDED_HEIGHT = SCREEN_HEIGHT * 0.75

interface TrackingDrawerProps {
  driverName: string
  etaMinutes: number
  deliveryAddress: string
}

export const TrackingDrawer: React.FC<TrackingDrawerProps> = ({
  driverName,
  etaMinutes,
  deliveryAddress,
}) => {
  // O valor compartilhado armazena o ponto de deslocamento vertical a partir do rodapé
  const translateY = useSharedValue(-COLLAPSED_HEIGHT)
  const context = useSharedValue({ y: 0 })

  const gesture = Gesture.Pan()
    .onStart(() => {
      context.value = { y: translateY.value }
    })
    .onUpdate(event => {
      // Limitar o arrastamento entre os limites expandido e colapsado
      translateY.value = Math.max(
        -EXPANDED_HEIGHT,
        Math.min(-COLLAPSED_HEIGHT, context.value.y + event.translationY)
      )
    })
}
```



```

    }
    onEnd() => {
        // Lógica de mola magnetica com base no ponto médio de deslocamento
        const middlePoint = (-EXPANDED_HEIGHT - COLLAPSED_HEIGHT) / 2
        if (translateY.value < middlePoint) {
            translateY.value = withSpring(-EXPANDED_HEIGHT, { damping: 15, stiffness: 90 })
        } else {
            translateY.value = withSpring(-COLLAPSED_HEIGHT, { damping: 15, stiffness: 90 })
        }
    }

    const animatedStyle = useAnimatedStyle() => {
        return {
            transform: [{ translateY: translateY.value }],
        }
    }

    const overlayOpacity = useAnimatedStyle() => {
        return {
            opacity: interpolate(
                translateY.value,
                [-COLLAPSED_HEIGHT, -EXPANDED_HEIGHT],
                [0, 0.4],
                Extrapolation.CLAMP
            ),
        }
    }

    return (
        <
            <Animated.View style={[styles.overlay, overlayOpacity]} pointerEvents="none" />
            <GestureDetector gesture={gesture} />
            <Animated.View style={[styles.drawer, animatedStyle]} />
            <View style={styles.handle} />
            <View style={styles.container} />
            <Text style={styles.statusText}>A sua entrega está a caminho</Text>
            <View style={styles.etaBadge} />
            <Text style={styles.etaText}>Chegada em {etaMinutes} min</Text>
        </View>
        <View style={styles.separator} />
        <Text style={styles.infoLabel}>Entregador Parceiro</Text>
        <Text style={styles.infoValue}>{driverName}</Text>
        <Text style={styles.infoLabel}>Morada de Destino</Text>
    )

```

```
        <Text style={styles.infoValue}>{deliveryAddress}</Text>
      </View>
    </Animated.View>
  </GestureDetector>
</>
</>
</>
```

```
const styles = StyleSheet.create({
  overlay: {
    ...StyleSheet.absoluteFillObject,
    backgroundColor: '#000000',
  },
  drawer: {
    position: 'absolute',
    top: SCREEN_HEIGHT,
    width: '100%',
    height: EXPANDED_HEIGHT,
    backgroundColor: '#ffffff',
    borderTopLeftRadius: 32,
    borderTopRightRadius: 32,
    shadowColor: '#000000',
    shadowOffset: { width: 0, height: -8 },
    shadowOpacity: 0.15,
    shadowRadius: 16,
    elevation: 8,
  },
  handle: {
    width: 48,
    height: 5,
    borderRadius: 2.5,
    backgroundColor: '#e5e/eb',
    alignSelf: 'center',
    marginVertical: 14,
  },
  container: {
    paddingHorizontal: 24,
    paddingTop: 8,
  },
  statusTitle: {
    fontSize: 22,
    fontWeight: '700',
    color: '#1f2937',
  },
});
```

```

    etaBadge:
      backgroundColor: #0284c7
      alignSelf: flex-start
      paddingVertical: 6
      paddingHorizontal: 12
      borderRadius: 8
      marginTop: 10
    etaText:
      color: #fff
      fontWeight: 600
      fontSize: 14
    separator:
      height: 1
      backgroundColor: #f3f4f6
      marginVertical: 20
    infoLabel:
      fontSize: 12
      fontWeight: 600
      color: #9ca3af
      textTransform: uppercase
      letterSpacing: 0.8
    infoValue:
      fontSize: 16
      fontWeight: 500
      color: #374151
      marginTop: 4
      marginBottom: 16
  
```

4. Next.js 15 Middleware Multilocatário (Routing & Isolation)

Este ficheiro intercede nas requisições HTTP dirigidas aos painéis web para assegurar que um administrador de uma franquia não consiga visualizar ou manipular dados correspondentes a outro inquilino²².

TypeScript

```
// apps/web-tenant-admin/src/middleware.ts
```

```
import { NextResponse } from 'next/server'
```

```
import type { NextRequest } from 'next/server'
```

```
import { createServerClient } from '@supabase/ssr'
```

```
export async function middleware(request: NextRequest)
```

```
  let response = NextResponse.next({
```

```
    request
```

```
    headers: request.headers,
```

```
  })
```

```
  //
```

```
  // Inicializar o cliente SSR para gerir sessões via cookies nativos
```

```
  const supabase = createServerClient
```

```
    process.env.NEXT_PUBLIC_SUPABASE_URL,
```

```
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
```

```
  )
```

```
  cookies
```

```
    getAll()
```

```
    return request.cookies.getAll().map(({ name, value }) => ({ name, value }))
```

```
  )
```

```
  setAll(cookiesToSet)
```

```
    cookiesToSet.forEach(({ name, value, options }) =>
```

```
      request.cookies.set({ name, value, ...options })
```

```
    )
```

```
  response = NextResponse.next({
```

```
    request
```

```
    headers: request.headers,
```

```
  })
```

```
  //
```

```
  cookiesToSet.forEach(({ name, value, options }) =>
```

```
    response.cookies.set({ name, value, ...options })
```

```
  )
```

```
  //
```

```
  //
```

```
  //
```

```
  //
```

```
  //
```

```

// Validar se o utilizador possui sessão ativa e autenticada
const (data session) = await supabase.auth.getSession()

const url = request.nextUrl.clone()
const path = url.pathname

// Redirecionamento se tentar aceder a rotas restritas sem autenticação
if (session && path.startsWith('/dashboard')) {
  url.pathname = '/login'
  return NextResponse.redirect(url)
}

if (session) {
  const userRole = session.user.app_metadata.user_role
  const userTenantId = session.user.app_metadata.tenant_id

  // Bloqueio de utilizadores normais (clientes) que tentem aceder ao painel administrativo
  if (path.startsWith('/dashboard') && userRole !== 'tenant_admin') {
    url.pathname = '/unauthorized'
    return NextResponse.redirect(url)
  }

  // Injetar o tenant_id da sessão como um cabeçalho HTTP interno para consultas subsequentes
  no servidor
  if (path.startsWith('/dashboard')) {
    const requestHeaders = new Headers(request.headers)
    requestHeaders.set('x-tenant-id', userTenantId || '')

    return NextResponse.next({
      request:
        headers: requestHeaders
    })
  }
}

return response

export const config = {
  matcher: '/dashboard/:path*' '/login'
}

```

Prompt de Engenharia de Geração para o Claude Code

Você é um Engenheiro de Software Principal especialista no ecossistema Turborepo, Next.js 15, Expo Router, Tailwind v4 e Supabase. Sua tarefa é codificar a versão de produção do ecossistema de distribuição de gás e água mineral multilocatário descrito nas diretrizes técnicas que se seguem.

1. ESTRUTURA DO MONOREPO

Construa e instancie a arquitetura do projeto seguindo a estrutura de diretórios do Turborepo:

- 'apps/web-tenant-admin' (Painel do franqueado em Next.js 15 App Router utilizando Tailwind v4, TypeScript e shadcn/ui).
- 'apps/web-platform-super' (Painel de administração global do SaaS em Next.js 15 para controle de inquilinos e faturamento).
- 'apps/mobile-customer' (Aplicação móvel Expo do cliente, habilitada com a Nova Arquitetura Fabric e Bridgeless mode).
- 'apps/mobile-driver' (Aplicação móvel Expo do entregador para controle logístico).
- 'packages/db' (Modelos de dados e esquemas do Drizzle ORM).
- 'packages/ui-web' (Componentes web baseados em React 19).
- 'packages/ui-native' (Estilização móvel baseada em NativeWind v4 e animações de alta fidelidade com Reanimated v4).
- 'packages/api-client' (Empacotador de chamadas do Supabase, com suporte a tipagem TypeScript robusta de ponta a ponta).

2. BASE DE DADOS E INFRAESTRUTURA SUPABASE

- Crie os ficheiros SQL de migração sob o esquema de integridade detalhado na especificação técnica.
- Configure todas as tabelas transacionais ('tenants', 'profiles', 'products', 'orders', 'order_items', 'cashback_ledger', 'driver_locations').
- Forneça a implementação de índices B-Tree e o índice geoespacial GiST ('ix_driver_locations_spatial') para PostGIS na localização de entregadores ativos.
- Implemente o trigger global de DDL Event Trigger para garantir que qualquer nova tabela criada receba a cláusula de proteção RLS ('force_rls_auto_enable') de forma automática.
- Crie e ligue o gatilho 'custom_access_token_hook' que lê o perfil e injeta de forma segura o 'tenant_id' e a 'user_role' no 'app_metadata' do JWT, de modo a acelerar em O(1) o desempenho de RLS do Postgres.
- Desenvolva as políticas RLS para todas as tabelas, aplicando a otimização de cache baseada na cláusula: 'USING (tenant_id = (SELECT (auth.jwt() -> "app_metadata" ->> "tenant_id")::uuid))'

3. MOTOR DE CASHBACK EM CÊNTIMOS

- Codifique funções de base de dados e de backend utilizando inteiros estritos em

cêntimos.

- Implemente rotas transacionais isoladas onde cada produto de gás ou água tenha a sua respetiva percentagem de cashback calculada em tempo real com base no valor faturado da ordem.
- O saldo do cliente deve ser obtido através de agregação sumária do livro de registos ('cashback_ledger').

4. RASTREAMENTO E LOGÍSTICA POSTGIS (REALTIME)

- No painel administrativo do inquilino ('apps/web-tenant-admin'), desenvolva a rota de despacho inteligente com base no operador de busca de vizinho mais próximo do Postgres ('<->'). Apresente os três entregadores fisicamente mais próximos do ponto do pedido que tenham carga ativa no seu veículo.
- Na aplicação móvel do cliente ('apps/mobile-customer'), integre as conexões WebSocket do Supabase Realtime para ouvir e projetar no mapa a localização instantânea emitida pela aplicação do entregador. Utilize o componente 'TrackingDrawer' com Reanimated v4 para transições contínuas a 60+ FPS baseadas inteiramente na thread nativa de UI.

5. CONTROLO E VALIDAÇÃO DE ENTREGAS (PROOF OF DELIVERY)

- Na aplicação do entregador ('apps/mobile-driver'), implemente o fluxo de transição de status ('accepted', 'in_transit', 'delivered').
- Force a verificação de duplo fator no local de destino: a) Validador de geofencing de proximidade física (o entregador deve estar a menos de 50 metros do destino da encomenda). b) Input do código alfanumérico OTP fornecido pelo cliente. c) Upload obrigatório de fotografia de confirmação no Supabase Storage sob regras restritas de visualização RLS.

Gere o código completo do monorepo, sem marcas de omissão, garantindo o máximo de tipagem de dados e conformidade estrita com todos os padrões arquiteturais de segurança e otimização.

Conclusões e Recomendações Estratégicas

A transição operacional de um ecossistema de postos de combustível para um modelo de distribuição e venda direta de gás e água exige um planeamento técnico rigoroso, focado no isolamento rigoroso de inquilinos e na resiliência de canais em tempo real¹⁰. Com base na análise das melhores práticas de arquitetura de dados e engenharia móvel, estabelecem-se as seguintes diretrizes para implementação¹⁰:

1. Estratégia de Provisionamento e Onboarding de Novos Inquilinos

O processo de criação de uma nova franquia na plataforma (por exemplo, a revendedora do pai do utilizador ou futuros clientes comerciais) deve ser inteiramente transacionado via código através de uma função de borda do Supabase (*Edge Function*)⁹. O fluxo deve ser executado utilizando a chave administrativa de sistema (*service_role*)²³ e estruturado da seguinte forma:

- Inserção do novo registo na tabela tenants com validação de CNPJ única⁹.
- Geração do perfil administrativo inicial do franqueado (profiles com role tenant_admin

associado ao tenant_id recém-criado)²⁵.

- Criação de categorias padrão de produtos (Ex: botijas de gás de 13kg e garrações de água de 20L) pré-configuradas com percentagens de cashback de referência¹¹.
- Configuração do balanço de cashback do inquilino para monitorização analítica no painel central do SaaS².

2. Limitação de Tráfego de Geolocalização (Throttling do GPS de Motoristas)

Manter atualizações em tempo real da frota de entregadores pode sobrecarregar a base de dados se as coordenadas forem enviadas de forma indiscriminada¹³. Recomenda-se implementar no código do cliente móvel (mobile-driver) uma rotina de filtragem híbrida baseada no acelerómetro do dispositivo:

- As atualizações de localização do motorista devem ser enviadas ao Supabase no máximo uma vez a cada 15 segundos se o veículo estiver em movimento¹³.
- Se o veículo parar (detectado pelo acelerómetro ou por diferença posicional inferior a 3 metros por minuto), o envio de dados deve ser automaticamente reduzido para intervalos de 3 minutos, poupando bateria do dispositivo do trabalhador e reduzindo em até 80% o tráfego de escritas na base de dados transacional¹⁰.

3. Regularização Fiscal e Legal no Transporte de Cilindros de Gás

Diferente da venda de combustíveis na pista, que ocorre num local comercial licenciado de forma definitiva¹, o transporte de recipientes transportáveis de gás liquefeito de petróleo (GLP) em veículos de entrega está sujeito a rigorosas regulamentações de segurança nacional (como as normas da ANP no Brasil ou regulamentações equivalentes da União Europeia).

A aplicação de entrega de gás deve incorporar salvaguardas operacionais cruciais no seu código de despacho¹⁰:

- O painel administrativo do franqueado (apps/web-tenant-admin) deve bloquear a liberação de entregas para motoristas cuja documentação de certificação para transporte de produtos perigosos esteja vencida.
- A aplicação do entregador (apps/mobile-driver) deve registar o peso total de gás transportado no início de cada turno de trabalho⁵. O sistema deve impedir a atribuição de novos cilindros ao motorista se a carga máxima de segurança permitida para a categoria do seu veículo de distribuição for ultrapassada, garantindo conformidade com a legislação de trânsito e segurança pública¹⁰.

A correta aplicação de todas as técnicas estruturadas neste relatório fornecerá uma plataforma corporativa altamente escalável, capaz de se defender de fraudes e proporcionar uma experiência tátil leve e fluida ao utilizador final⁹.

Referências citadas

1. Sobre Nós - Ecossistema Xulis, <https://xulis.com.br/sobre-nos/>
2. FAQ - Xulis, <https://xulis.com.br/faq/>

3. Xulis: Pagamentos e Cashback - Apps on Google Play, <https://play.google.com/store/apps/details?id=com.xuliz>
4. Ecosystema Xulis: Sistema Para Postos de Combustíveis, <https://xulis.com.br/>
5. Maquininhas Xulis, <https://xulis.com.br/maquininhas-xulis/>
6. ERP Xulis, <https://xulis.com.br/erp-xulis/>
7. Xulis: Pagamentos e Cashback – Apps no Google Play, <https://play.google.com/store/apps/details?id=com.xuliz&hl=pt>
8. Aplicativo - Ecosystema Xulis, <https://xulis.com.br/aplicativo/>
9. Supabase for B2B SaaS, <https://supabase.com/solutions/b2b-saas>
10. Fuel Delivery App Development Guide: Features & Cost, <https://www.xbytesolutions.com/fuel-delivery-app-development-guide/>
11. How to Develop a Fuel Delivery App: Cost, Features & Tech Stack (2025–2026 Guide), <https://medium.com/@princy.icoderz/how-to-develop-a-fuel-delivery-app-cost-features-tech-stack-2025-2026-guide-32cfe60d8c7b>
12. Fuel Delivery App Development: A Complete Guide - Alpha Information Systems, <https://www.aalpha.net/articles/how-to-develop-an-on-demand-fuel-delivery-app/>
13. Booster Fuels Business Model: Mobile Fuel Delivery App Development Guide 2026, <https://nectarbits.ca/blog/booster-fuel-delivery-app-business-model/>
14. Contas a pagar - Ecosystema Xulis, <https://xulis.com.br/contas-a-pagar/>
15. Supabase Row Level Security Explained With Real Examples | by JigsDev - Medium, <https://medium.com/@jigsz6391/supabase-row-level-security-explained-with-real-examples-6d06ce8d221c>
16. How On-Demand Gas Delivery Apps Work: A Complete Startup Guide - Coherent Lab, <https://www.coherentlab.com/blog/how-on-demand-gas-delivery-apps-work>
17. Connect your Turbostarter (Next.js, React Native, Vite) app to Supabase in 5 minutes, <https://www.turbostarter.dev/blog/connect-your-turbostarter-nextjs-react-native-vite-app-to-supabase-in-5-minutes>
18. New Supabase Docs, built with Next.js, <https://supabase.com/blog/new-supabase-docs-built-with-nextjs>
19. 15 Best T3 Stack Templates & Starters 2026 - AdminLTE.IO, <https://adminlte.io/blog/t3-stack-templates/>
20. Monorepo with Next.js and Expo - Convex, <https://www.convex.dev/templates/monorepo>
21. How to Create Fluid Animations with React Native Reanimated v4 - freeCodeCamp, <https://www.freecodecamp.org/news/how-to-create-fluid-animations-with-react-native-reanimated-v4/>
22. Supabase & Next.js App Router Starter Template for Auth - Vercel, <https://vercel.com/templates/next.js/supabase>

23. Supabase RLS Best Practices: Production Patterns for Secure Multi-Tenant Apps - MakerKit, <https://makerkit.dev/blog/tutorials/supabase-rls-best-practices>
24. Row Level Security | Supabase Docs, <https://supabase.com/docs/guides/database/postgres/row-level-security>
25. Custom Claims & Role-based Access Control (RBAC) | Supabase Docs, <https://supabase.com/docs/guides/api/custom-claims-and-role-based-access-control-rbac>
26. Build a RAG App with Descope, Supabase, and pgvector: Part 2, <https://www.descope.com/blog/post/rag-descope-supabase-pgvector-2>
27. Best Practices for Supabase | Security, Scaling & Maintainability - Leanware, <https://leanware.co/insights/supabase-best-practices>
28. Mastering UI Animations in React Native Using Reanimated — A Practical Guide, <https://dev.to/saloniagrawal/mastering-ui-animations-in-react-native-using-reanimated-a-practical-guide-2404>
29. How to Create Smoother User Experiences with React Native - Unosquare, <https://www.unosquare.com/blog/how-to-create-smoother-user-experiences-with-react-native/>
30. How to Implement Smooth Animations with React Native Reanimated - OneUptime, <https://oneuptime.com/blog/post/2026-01-15-react-native-reanimated/view>
31. Exploring React Native's Animation Capabilities with Reanimated - MetaDesign Solutions, <https://metadesignsolutions.com/blog/exploring-react-natives-animation-capabilities-with-reanimated>
32. Token Security and Row Level Security | Supabase Docs, <https://supabase.com/docs/guides/auth/oauth-server/token-security>
33. Supabase Custom Claims - DEV Community, <https://dev.to/supabase/supabase-custom-claims-34l2>
34. How to implement custom claims with Supabase - GitHub, <https://github.com/supabase-community/supabase-custom-claims>
35. Políticas de privacidade - Ecossistema Xulis, <https://xulis.com.br/politicas-de-privacidade/>